

Resoluções Didáticas e Científicas —

Capítulo 1

Obra: C: Como Programar, 6ª Edição

Autores: Paul Deitel & Harvey Deitel

Abordagem: Progressão Incremental e Rigor de Engenharia de Software

Apresentação Pedagógica

Olá! É um prazer apresentar o guia oficial de soluções para o Capítulo 1. Em conformidade com o nosso método pedagógico incremental, restringimos as explicações e abordagens estritamente aos conceitos fundamentais de arquitetura de computadores, evolução das linguagens de programação e ao ambiente de desenvolvimento da linguagem C. Nenhuma estrutura de controle ou sintaxe avançada dos capítulos posteriores será antecipada aqui, garantindo um alicerce sólido para o seu aprendizado.

Abaixo, você encontrará a resolução científica completa formatada para visualização direta e, ao final, o código-fonte LaTeX integral estruturado para compilação profissional.

1. Exercícios de Autorrevisão

Exercício 1.1 — Fundamentos de Computação

- a) A empresa que popularizou a computação pessoal foi a **Apple**.
- b) O computador que tornou a computação pessoal possível no comércio e na indústria foi o **IBM Personal Computer**.
- c) Computadores processam dados sob o controle de conjuntos de instruções chamadas **programas de computador**.
- d) As seis unidades lógicas do computador são: **unidade de entrada, unidade de saída, unidade de memória, unidade lógica e aritmética (ALU), unidade central de processamento (CPU) e unidade de armazenamento secundária**.
- e) Os três tipos de linguagem que discutimos são: **linguagens de máquina, linguagens simbólicas (assembly) e linguagens de alto nível**.
- f) Os programas que traduzem programas em linguagem de alto nível na linguagem de máquina são chamados de **compiladores**.
- g) C é bastante conhecida como a linguagem de desenvolvimento do sistema operacional **UNIX**.
- h) O Department of Defense (Departamento de Defesa) desenvolveu a linguagem Ada com uma capacidade chamada de **multitarefa** que permite que os programadores especifiquem as atividades que podem ocorrer em paralelo.

Exercício 1.2 — O Ambiente de Desenvolvimento C

- a) Programas em C são normalmente digitados em um computador utilizando-se um programa **editor**.

- b)** Em um sistema C, um programa **pré-processador** é executado automaticamente antes que se inicie a fase de tradução.
- c)** Os tipos mais comuns de diretivas de pré-processador são utilizados para **incluir outros arquivos no arquivo a ser compilado** e **substituir símbolos especiais por texto de programa**.
- d)** O programa **editor de ligação (ou linker)** combina a saída do compilador com várias funções de biblioteca para produzir uma imagem executável.
- e)** O programa **carregador (loader)** transfere a imagem executável do disco para a memória.
- f)** Para carregar e executar o programa compilado mais recentemente em um sistema Linux, digite `./a.out`.

2. Exercícios Regulares

Exercício 1.3 — Classificação de Componentes (Hardware vs. Software)

A distinção clara entre os componentes físicos tangíveis (hardware) e as instruções lógicas e programas executados por esses componentes (software) é crucial para a engenharia de sistemas.

Item	Classificação	Justificativa Científica
a) CPU	Hardware	É a Unidade Central de Processamento física, constituída por circuitos integrados e silício.
b) Compilador C++	Software	É um programa executável utilitário que traduz código de alto nível em código de máquina.
c) ALU	Hardware	A Unidade Lógica e Aritmética é um circuito digital físico interno integrado à CPU.
d) Pré-processador C++	Software	É um componente de software estrutural que manipula o código-fonte antes da compilação.
e) Unidade de entrada	Hardware	Representa dispositivos físicos (ex: teclado, mouse) usados para capturar dados externos.
f) Um programa editor	Software	É uma aplicação lógica usada

Item	Classificação	Justificativa Científica
		para a criação e modificação de arquivos de texto.

Exercício 1.4 — Análise de Independência de Máquina

Por que escrever em uma linguagem independente de máquina em vez de dependente da máquina?

A principal razão é a **portabilidade do software** e a eficiência no desenvolvimento. Escrever em uma linguagem independente de máquina (como C ou Java) permite que o mesmo código-fonte seja compilado e executado em diversas arquiteturas de hardware diferentes com alterações mínimas ou nulas. Isso reduz drasticamente o tempo e o custo de engenharia, maximizando o alcance do sistema de software.

Por que uma linguagem dependente de máquina poderia ser mais apropriada para certos programas?

Linguagens dependentes de máquina (como Linguagem de Máquina pura ou Assembly) fornecem **controle total e direto sobre o hardware subjacente**. Elas são fundamentais para o desenvolvimento de sistemas onde o desempenho em tempo real é crítico, para a escrita de drivers de dispositivos que precisam acessar registradores físicos específicos do processador, ou para sistemas embarcados de altíssima restrição de memória, onde cada instrução física deve ser controlada milimetricamente.

Exercício 1.5 — Arquitetura e Fluxo de Dados do Computador

- a) Unidade lógica que recebe informações de fora: **Unidade de entrada.**
- b) O processo de instruir o computador para resolver problemas: **Programação de computadores.**
- c) Linguagem que usa abreviações em inglês para instruções de máquina: **Linguagem simbólica (ou Assembly).**
- d) Unidade que envia informações processadas para fora: **Unidade de saída.**
- e) Unidades lógicas que retêm informações: **Unidade de memória (primária) e unidade de armazenamento secundária.**
- f) Unidade lógica que realiza cálculos: **Unidade lógica e aritmética (ALU).**
- g) Unidade lógica que toma decisões lógicas: **Unidade lógica e aritmética (ALU).**
- h) Nível de linguagem mais conveniente para elaboração rápida de programas: **Linguagem de alto nível.**
- i) A única linguagem que o computador entende nativamente com clareza: **Linguagem de máquina.**
- j) Unidade lógica que coordena as atividades de todas as outras: **Unidade central de processamento (CPU)** (gerenciada por sua Unidade de Controle).

Exercício 1.6 — Verificação de Sentenças

a) Linguagens de máquina geralmente são dependentes da máquina.

Verdadeiro. O conjunto de instruções binárias nativas de um processador é projetado de forma fixa em seus circuitos eletrônicos. Consequentemente, o código de máquina de uma arquitetura

(como x86) não funciona nativamente em outra (como ARM).

b) Assim como outras linguagens de alto nível, C geralmente é considerada como independente da máquina.

Verdadeiro. Um código escrito em conformidade com o padrão C ISO pode ser portado para praticamente qualquer plataforma de hardware contemporânea, desde que o sistema de destino possua um compilador C compatível para gerar o binário específico daquela máquina.

Exercício 1.7 — Análise de Canais de Fluxo de Dados (I/O Padrão)

No ambiente C e sistemas operacionais modernos, os dados fluem através de canais padronizados denominados *streams*:

- **a) stdin (Fluxo de Entrada Padrão):** É a origem de dados padrão de um programa. Cientificamente, representa uma abstração orientada a fluxo que, por definição do sistema operacional, está vinculada ao teclado, permitindo a captura de dados de entrada do usuário.
- **b) stdout (Fluxo de Saída Padrão):** É o destino padrão para exibição dos dados gerados com sucesso pela execução normal do programa. Geralmente, está direcionado e conectado ao monitor ou console de comando.
- **c) stderr (Fluxo de Erro Padrão):** É um canal de saída independente dedicado exclusivamente ao reporte de mensagens de erro, falhas do sistema e diagnósticos. Ele é intencionalmente separado do stdout para que o usuário ou scripts automatizados possam redirecionar a saída de dados principal sem perder os alertas críticos de erro, embora por padrão também seja exibido na tela.

Exercício 1.8 — Engenharia de Software e a Programação Orientada a Objetos (OOP)

O foco massivo na programação orientada a objetos na engenharia de software moderna deve-se à sua capacidade de resolver a crise de complexidade do software de grande porte. A OOP introduz uma mentalidade centrada na **reutilização de software** e na modularidade. Ao mapear entidades do mundo real em componentes de código chamados *objetos* — que integram intimamente seus dados (atributos) e comportamentos (métodos) — a OOP promove o encapsulamento, a manutenibilidade e a extensibilidade do sistema. Isso permite criar aplicações muito mais robustas, fáceis de depurar e flexíveis a mudanças corporativas em comparação com as limitações dos sistemas puramente estruturados tradicionais de grande escala.

Exercício 1.9 — Evolução das Linguagens de Programação

Descrição Histórica / Técnica	Linguagem Correspondente
a) Desenvolvida pela IBM para aplicações científicas e de engenharia.	FORTRAN
b) Desenvolvida especificamente para aplicações comerciais.	COBOL

Descrição Histórica / Técnica	Linguagem Correspondente
c) Desenvolvida para o ensino de programação estruturada.	Pascal
d) Seu nome homenageia a primeira pessoa a programar um computador no mundo.	Ada
e) Desenvolvida para familiarizar iniciantes com as técnicas de programação.	BASIC
f) Desenvolvida especificamente para ajudar os programadores a migrar para .NET.	Visual C#
g) Conhecida como a linguagem de desenvolvimento do UNIX.	C
h) Formada, principalmente, pela inclusão da programação orientada a objetos à linguagem C.	C++
i) Inicialmente, teve sucesso devido à sua capacidade de criar páginas Web com conteúdo dinâmico.	Java

3. Seção Fazendo a Diferença

Exercício 1.10 — Test-Drive: Calculadora de Emissão de Carbono

Fórmulas Científicas e Modelagem de Dados:

O cálculo do impacto ecológico individual (pegada de carbono) é quantificado em toneladas de CO₂ equivalente (tCO₂e) por ano. As equações estruturais baseiam-se na multiplicação do consumo bruto de recursos por fatores de emissão oficiais fornecidos por órgãos ambientais (como o EPA ou o IPCC):

Emissão Total Anual = (Consumo Elétrico [kWh] * Fator_Eletricidade) + (Gás Consumido [m³] * Fator_Gás) + (Combustível Auto [L] * Fator_Gasolina) + (Quilômetros de Voos * Fator_Aéreo)

Análise Incremental para o Desenvolvimento Futuro:

Para preparar a lógica algorítmica sem violar a restrição de escopo do Capítulo 1, mapeamos as necessidades estruturais:

- **Entradas necessárias:** Variáveis quantitativas numéricas para armazenar consumos mensais de utilitários e transporte.
- **Processamento:** Aplicação sequencial de coeficientes multiplicadores constantes lineares.
- **Saídas:** Apresentação do indicador de impacto ambiental acumulado no console.

Exercício 1.11 — Test-Drive: Calculadora de Índice de Massa Corporal (IMC)

Fórmulas Científicas de Cálculo de Saúde Pública:

O Índice de Massa Corporal baseia-se na relação antropométrica entre a massa e a estatura do indivíduo. A modelagem matemática varia estritamente com base no sistema de unidades selecionado:

Sistema Métrico Internacional (SI):

$IMC = \text{peso_em_kg} / (\text{altura_em_metros} * \text{altura_em_metros})$

Sistema Imperial (Unidades de Engenharia dos EUA):

$IMC = (\text{peso_em_libras} * 703) / (\text{altura_em_polegadas} * \text{altura_em_polegadas})$

Análise Incremental e Mapeamento Arquitetural:

Em capítulos posteriores, utilizaremos essa base científica para receber dados interativamente do usuário através da função `scanf`, realizaremos as operações aritméticas elementares e empregaremos operadores relacionais integrados à instrução `if` para classificar a faixa de saúde de acordo com os critérios estabelecidos pela Organização Mundial da Saúde (OMS).

Exercício 1.12 — Neutralidade de Gênero (Análise Algorítmica Manual)

Procedimento Passo a Passo (Especificação do Algoritmo Sequencial):

1. **Inicialização:** Posicione o dedo ou foco visual no início do primeiro parágrafo impresso, apontando para a primeira palavra.
2. **Leitura Atenta:** Leia a palavra corrente isolando-a de pontuações adjacentes.
3. **Pesquisa na Tabela de Correspondências:** Busque a palavra exata lida na coluna de termos de "gênero específico" da lista de referência previamente fornecida.
4. **Decisão e Substituição:**
 - *Caso seja encontrada na lista:* Risque a palavra original no papel e escreva imediatamente acima dela a palavra substituta correspondente de "gênero neutro" (ex: se ler "esposa", troque por "cônjuge").
 - *Caso não seja encontrada:* Mantenha a palavra intacta sem realizar modificações.
5. **Avanço de Fluxo:** Mova o foco visual para a próxima palavra à direita. Se atingir o término da linha corrente, desloque-se para o início da linha imediatamente inferior.
6. **Condição de Término:** Repita os passos 2, 3, 4 e 5 consecutivamente até que todas as palavras do parágrafo tenham sido verificadas. O processo encerra ao atingir o ponto final do texto.

Tratamento Científico de Termos Compostos e Prefixos (Caso "super-homem"):

Se o algoritmo processar estritamente correspondências de palavras inteiras isoladas, o termo composto "super-homem" seria ignorado por não ser uma cópia literal do token isolado "homem". Para garantir robustez técnica, o procedimento precisa empregar uma etapa de **análise de subcadeias de caracteres (substring match)**. Sob essa ótica, o termo composto seria temporariamente segmentado; o radical ou sufixo identificado ("homem") dispararia a regra de tradução cadastrada, sendo mapeado para "pessoa", enquanto o prefixo ("super-") e o caractere delimitador (hífen) seriam preservados integralmente. Assim, o resultado manual gerado com precisão formal seria "**super-pessoa**".

4. Código-Fonte LaTeX Completo para Compilação

Copie e cole o código técnico abaixo em qualquer editor ou compilador LaTeX (como Overleaf, TeXworks ou TeXstudio) para gerar os seus arquivos acadêmicos finais (.tex, .pdf) de forma profissional:

```
\documentclass[12pt]{article}
\usepackage[utf8]{inputenc}
\usepackage[brazil]{babel}
\usepackage{amsmath}
\usepackage{amsfonts}
\usepackage{booktabs}
\usepackage{geometry}
\geometry{a4paper, margin=1in}

\title{Resolu\c{c}\~oes Completas e Did\~aticas -- Cap\~itulo 1\\
\large C: Como Programar (6\textordfeminine\ Ed.\ Deitel)}
\author{Prof. Paul Deitel}
\date{\today}

\begin{document}
\maketitle

\section{Exerc\~icios de Autorrevis\~ao}

\subsection*{Exerc\~icio 1.1 -- Preenchimento de Lacunas (Arquitetura
e Hist\~orico)}
\begin{enumerate}
\item[a)] A empresa que popularizou a computa\c{c}\~ao pessoal foi
a \textbf{Apple}.
\item[b)] O computador que tornou a computação pessoal possível no
comércio e na indústria foi o \textbf{IBM Personal Computer}.
\item[c)] Computadores processam dados sob o controle de conjuntos
de instruções chamadas \textbf{programas de computador}.
\item[d)] As seis unidades lógicas do computador são:
\textbf{unidade de entrada, unidade de saída, unidade de memória,
unidade lógica e aritmética (ALU), unidade central de processamento
(CPU) e unidade de armazenamento secundária}.
\item[e)] Os três tipos de linguagem que discutimos são:
\textbf{linguagens de máquina, linguagens simbólicas (assembly) e
linguagens de alto nível}.
\item[f)] Os programas que traduzem programas em linguagem de alto
nível na linguagem de máquina são chamados de \textbf{compiladores}.
\item[g)] C é bastante conhecida como a linguagem de
desenvolvimento do sistema operacional \textbf{UNIX}.
\item[h)] O Department of Defense desenvolveu a linguagem Ada com
uma capacidade chamada de \textbf{multitarefa}.
\end{enumerate}

\subsection*{Exerc\~icio 1.2 -- O Ambiente de Desenvolvimento C}
\begin{enumerate}
```

\item[a)] Programas em C são normalmente digitados em um computador utilizando-se um programa \textbf{editor}.

\item[b)] Em um sistema C, um programa \textbf{pré-processador} é executado automaticamente antes que se inicie a fase de tradução.

\item[c)] Os tipos mais comuns de diretivas de pré-processador são utilizados para \textbf{incluir outros arquivos no arquivo a ser compilado} e \textbf{substituir símbolos especiais por texto de programa}.

\item[d)] O programa \textbf{editor de ligação (ou linker)} combina a saída do compilador com várias funções de biblioteca para produzir uma imagem executável.

\item[e)] O programa \textbf{carregador} transfere a imagem executável do disco para a memória.

\item[f)] Para carregar e executar o programa compilado mais recentemente em um sistema Linux, digite \textbf{./a.out}.

\end{enumerate}

\newpage

\section{Exercícios Regulares}

\subsection{Exercício 1.3 -- Classificação (Hardware vs. Software)}

\begin{table}[h!]

\centering

\caption{Classificação de Componentes do Sistema Computacional}

\begin{tabular}{lll}

\toprule

\textbf{Item} & \textbf{Classificação} & \textbf{Justificativa Teórica} \\

\midrule

a) CPU & Hardware & Unidade física central de processamento de silício. \\

b) Compilador C++ & Software & Programa utilitário tradutor de código-fonte. \\

c) ALU & Hardware & Circuito interno de execução aritmética da CPU. \\

d) Pré-processador C++ & Software & Aplicação que processa diretivas de compilação. \\

e) Unidade de entrada & Hardware & Dispositivos periféricos físicos de captura de dados. \\

f) Um programa editor & Software & Aplicação utilitária de edição de arquivos de texto. \\

\bottomrule

\end{tabular}

\end{table}

\subsection{Exercício 1.4 -- Independência de Máquina}

A escrita de programas em linguagens de alto nível independentes de máquina provê \textbf{portabilidade}. Isso implica que o software

desenvolvido pode ser reutilizado e executado em variadas plataformas de hardware por meio de uma simples recompilação, otimizando o esforço de desenvolvimento.\

Por outro lado, linguagens dependentes de máquina (Assembly/Máquina) são cruciais quando se faz estritamente necessário o controle de baixo nível e acesso direto aos registradores do hardware, como no desenvolvimento de kernels de sistemas operacionais, rotinas críticas de temporização em tempo real e drivers de dispositivos periféricos de alta performance.

```
\subsection*{Exercício 1.5 -- Arquitetura de Fluxo de Dados}
\begin{tabular}{ll}
a) Unidade de entrada & f) Unidade lógica e aritmética (ALU) \\
b) Programação de computadores & g) Unidade lógica e aritmética (ALU) \\
c) Linguagem simbólica (Assembly) & h) Linguagem de alto nível \\
d) Unidade de saída & i) Linguagem de máquina \\
e) Memória primária e secundária & j) Unidade central de processamento (CPU)
\end{tabular}
```

```
\subsection*{Exercício 1.6 -- Falso ou Verdadeiro}
\begin{enumerate}
\item[a)] \textbf{Verdadeiro.} As instruções binárias são diretamente implementadas nos circuitos lógicos específicos daquela família de processadores.
\item[b)] \textbf{Verdadeiro.} O código-fonte padrão ISO C pode ser portado e compilado em qualquer arquitetura de hardware dotada de um compilador compatível.
\end{enumerate}
```

```
\subsection*{Exercício 1.7 -- Conceitos de Streams de I/O}
\begin{itemize}
\item \textbf{stdin:} Fluxo de entrada padrão abstrato que, por convenção de engenharia, captura dados enviados via teclado.
\item \textbf{stdout:} Fluxo de saída padrão abstrato destinado à exibição da saída padrão do programa na tela ou terminal.
\item \textbf{stderr:} Fluxo de erro padrão independente, usado especificamente para mensagens críticas de diagnóstico, isolado do fluxo de dados comum.
\end{itemize}
```

```
\subsection*{Exercício 1.8 -- Justificativa da Programação Orientada a Objetos (OOP)}
```

A atenção dada à OOP reside na alta capacidade de \textbf{reutilização de software} e contenção da complexidade. Ao encapsular dados e comportamentos em entidades modulares denominadas objetos, os sistemas

tornam-se consideravelmente mais fáceis de estender, depurar e manter a longo prazo nas corporações.

```
\subsection*{Exercício 1.9 -- Identificação de Linguagens}
\begin{tabular}{lll}
a) FORTRAN & d) Ada & g) C \\
b) COBOL & e) BASIC & h) C++ \\
c) Pascal & f) Visual C\# & i) Java
\end{tabular}
```

```
\newpage
\section{Seção -- Fazendo a Diferença}
```

Exercício 1.10 -- Calculadora de Emissão de Carbono
O cálculo cientificamente embasado para mensurar as emissões de gases de efeito estufa é expresso por meio da soma ponderada de consumos utilitários lineares:

```
\begin{equation*}
E_{\text{Total}} = (C_{\text{Elet}} \times f_{\text{Elet}}) +
(C_{\text{Gás}} \times f_{\text{Gás}}) + (C_{\text{Auto}} \times
f_{\text{Auto}}) + (D_{\text{Voo}} \times f_{\text{Voo}})
\end{equation*}
```

onde C denota as variáveis quantitativas e f os coeficientes fixos de emissão. De forma incremental para o escopo do Capítulo 1, definimos o escopo lógico do programa que futuramente processará dados escalares sequenciais para produzir o relatório de emissão em formato métrico anual.

Exercício 1.11 -- Calculadora de IMC
O Índice de Massa Corporal modela matematicamente o status ponderal do indivíduo através das seguintes equações de saúde pública, separadas por sistemas de medição:

```
\begin{equation*}
\text{IMC (Métrico)} = \frac{\text{peso (kg)}}{\text{altura (m)}^2}
\quad \text{IMC (Imperial)} = \frac{\text{peso (lbs)}}
{\text{altura (in)}^2}
\end{equation*}
```

No Capítulo 2, traduziremos esta formulação teórica para código C estruturado utilizando armazenamento de ponto flutuante de precisão simples para aquisição de entradas e geração de resultados.

Exercício 1.12 -- Algoritmo de Neutralidade de Gênero

O procedimento formal sequencial de processamento de texto manual consiste nas seguintes etapas lógicas estruturadas:

```
\begin{enumerate}
\item Posicione o ponteiro de leitura no caractere inicial do
texto.
```

\item Isole a palavra corrente contida entre os espaços em branco ou pontuações.

\item Realize uma busca linear na tabela de mapeamento de termos de gênero específico fornecida.

\item Se houver um acerto completo ou parcial (substring match), aplique a substituição pelo equivalente neutro. Caso contrário, mantenha o termo original estável.

\item Avance o ponteiro para a próxima palavra subsequente.

\item Repita até identificar o marcador do final do parágrafo.

\end{enumerate}

\textbf{Análise do Caso Especial ``super-homem':} Visando a precisão do engenheiro de software, o processador textual manual deve empregar correspondência por subcadeias de caracteres (\textit{substring match}). Ao detectar o sufixo ou radical composto ``homem'', este é mapeado para o neutro ``pessoa'', mantendo o prefixo e o delimitador hifenizado originais inviolados, culminando no termo preciso \textbf{super-pessoa}.

\end{document}